

單元 2：從 App Engine NDB 遷移至 Cloud NDB

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-2-cloudndb?hl=zh-tw>

步驟數：8

瞭解如何將簡易的 App Engine 應用程式從 ndb 遷移至 Cloud NDB。

目錄

1. [總覽](#)
2. [背景](#)
3. [設定/準備工作](#)
4. [更新設定檔 \(新增 Cloud NDB 程式庫\)](#)
5. [更新應用程式檔案](#)
6. [摘要/清除](#)
7. [加分題：遷移至 Python 3](#)
8. [其他資源](#)

1. 總覽

本系列程式碼研究室 (可自行調整步調的實作教學課程) 旨在協助 [Google App Engine](#) (標準) 開發人員完成一系列遷移作業，進而將應用程式現代化。最重要的步驟是捨棄原始的執行階段隨附服務，因為[新一代執行階段](#)更具彈性，可為使用者提供更多服務選項。改用新一代執行階段後，您就能更輕鬆地與 Google Cloud 產品整合、使用更多支援的服務，以及支援目前的語言版本。

本教學課程說明如何從 App Engine 內建的 [ndb \(Next Database\)](#) 用戶端程式庫遷移至 [Cloud NDB 用戶端程式庫](#)。

注意：如果您的應用程式未使用 [ndb](#) 或 [Datastore](#)，可以透過本程式碼研究室練習，熟悉這項遷移作業。

課程內容

- 使用 App Engine [ndb](#) 程式庫 (如果您不熟悉的話)
- 從 [ndb](#) 遷移至 Cloud NDB
- 進一步將應用程式遷移至 Python 3

軟硬體需求

- 具有下列項目的 [Google Cloud Platform 專案](#)：
 - [有效的 GCP 帳單帳戶](#)
 - [已啟用 App Engine](#)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發](#)及[部署](#) App Engine 應用程式的基本知識
- 可正常運作的 [Module 1 App Engine 應用程式](#)

問卷調查

步驟 2 · 約 1 分鐘

2. 背景

在單元 1 中，我們將網頁架構從 App Engine 的內建 `webapp2` 遷移至 `Flask`。在本程式碼研究室中，我們將繼續擺脫 App Engine 的內建服務，從 App Engine 的 `ndb` 程式庫切換至 Google Cloud NDB。

完成遷移作業後，您就能：

1. 遷移至 **Python 3** 和新一代 App Engine 執行階段
2. 遷移至 **Cloud Datastore** (適用於非 App Engine 應用程式的用戶端程式庫)
3. 將 **Python 2 (或 3) 應用程式容器化**，然後遷移至 **Cloud Run**
4. 新增使用 App Engine (發送) 工作佇列，然後遷移至 **Cloud Tasks**

但我們尚未達到這個目標。請先完成本程式碼研究室，再考慮這些後續步驟。本教學課程的遷移作業主要包含下列步驟：

1. 設定/準備工作
 2. 新增 Cloud NDB 程式庫
 3. 更新應用程式檔案
-

3. 設定/準備工作

在開始進行本教學課程的主要部分之前，請先設定專案、取得程式碼，然後部署基準應用程式，確保我們從可運作的程式碼開始。

1. 設定專案

如果您已完成[第 1 模組程式碼研究室](#)，建議重複使用該專案 (和程式碼)。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。確認專案已啟用計費帳戶，且已啟用 App Engine。

2. 取得基準範例應用程式

其中一項必要條件是擁有可運作的第 1 課範例應用程式。如果您已完成該教學課程，請使用您的解決方案。您可以立即完成 (上方連結)，也可以選擇略過，然後複製模組 1 存放區 (下方連結)。

無論您使用自己的程式碼或我們的程式碼，我們都會從「模組 1」的程式碼開始。本程式碼研究室 (第 2 模組) 會逐步說明每個步驟，完成後應會類似於 FINISH 點的程式碼 (包括從 Python 2 到 3 的選用「加分」移植作業)：

- 開始：[第 1 單元程式碼](#)
- 完成：模組 2「[Python 2 程式碼](#)」(加分題：[Python 3 程式碼](#))
- [整個存放區](#) (複製或下載 ZIP 檔案)

您的「STARTing Module 1」程式碼資料夾應包含下列內容：

```
$ ls
README.md          appengine_config.py  requirements.txt
app.yaml           main.py              templates
```

如果您已完成第 1 模組教學課程，也會有包含 Flask 及其依附元件的 `lib` 資料夾。如果沒有 `lib` 資料夾，請使用 `pip install -t lib -r requirements.txt` 指令建立，以便在下一個步驟中部署這個基準應用程式。如果您同時安裝了 Python 2 和 3，建議使用 `pip2`，而非 `pip`，以免與 Python 3 混淆。

3. (重新) 部署模組 1 應用程式

請立即執行剩餘的準備步驟：

1. 重新熟悉 `gcloud` 指令列工具 (如有必要)
2. (重新)將模組 1 程式碼部署至 App Engine (如有必要)。

成功執行這些步驟並確認運作正常後，我們將繼續本教學課程，從設定檔開始。

4. 更新設定檔 (新增 Cloud NDB 程式庫)

許多原本的 App Engine 內建服務都已發展成獨立產品，Datastore 就是其中之一。如今非 **App Engine** 應用程式也能使用 **Cloud Datastore**。對於長期使用 `ndb` 的使用者，Google Cloud 團隊建立了 Cloud NDB 用戶端程式庫，可與 Cloud Datastore 通訊。適用於 Python 2 和 3。

請更新確認檔案，將 App Engine `ndb` 換成 Cloud NDB，然後修改應用程式。

1. 更新「`requirements.txt`」

在第 1 模組中，我們應用程式的唯一外部依附元件是 Flask。現在要新增 Cloud NDB。以下是第 1 模組結束時的 `requirements.txt` 檔案：

- 更新前：

```
Flask==1.1.2
```

如要從 App Engine `ndb` 遷移，必須使用 Cloud NDB 程式庫 (`google-cloud-ndb`)，因此請將其套件新增至 `requirements.txt`。

- AFTER：

```
Flask==1.1.2
google-cloud-ndb==1.7.1
```

撰寫本程式碼研究室時，建議使用的最新版本為 1.7.1，但存放區中的 `requirements.txt` 可能有更新的版本。建議使用各程式庫的最新版本，但如果無法正常運作，可以回溯至舊版。

如果已有 `lib` 資料夾，且並非在上述步驟中建立，請刪除該資料夾。現在請使用 `pip install -t lib -r requirements.txt` 指令 (重新) 安裝更新後的程式庫，並視需要使用 `pip2` 而非 `pip`。

疑難排解：如果應用程式發生 `ImportError: cannot import name collections_abc` 錯誤，請確認 `lib` 中的 Cloud NDB 依附元件 `google-cloud-core` 至少為 1.4.4 版。導致該錯誤的**核心問題**只存在於 1.4.2 和 1.4.3 版本。

2. 更新「`app.yaml`」

新增 Google Cloud 用戶端程式庫 (例如 `google-cloud-ndb`) 時，必須符合幾項規定，這些規定都與納入「內建」程式庫有關，也就是 Google 伺服器上已提供的第三方套件。您不會在 `requirements.txt` 中列出這些檔案，也不會使用 `pip install` 複製這些檔案。唯一規定：

- 在 `app.yaml` 中指定內建程式庫
- 向他們指出可能使用的已複製第三方案式庫 (位於 `lib`)

以下是第 1 堂課的 `app.yaml`：

- 更新前：

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app
```

現在，請在新的 `libraries` 區段中，將下列程式碼新增至 `app.yaml`，以參照一對第三方組合套件：`grpcio` 和 `setuptools`：

```
libraries:
- name: grpcio
  version: 1.0.0
- name: setuptools
  version: 36.6.0
```

為何要使用這些內建程式庫？[gRPC](#) 是開放式 RPC 架構，所有 [Google Cloud 用戶端程式庫](#) (包括 `google-cloud-ndb`) 都會使用這個架構。`grpcio` 程式庫是 Python gRPC 介面卡，因此必須安裝。我們稍後會說明加入 `setuptools` 的原因。

- AFTER：

完成上述變更後，更新後的 `app.yaml` 應如下所示：

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app

libraries:
- name: grpcio
  version: 1.0.0
- name: setuptools
  version: 36.6.0
```

3. 更新「`appengine_config.py`」

`pkg_resources` 工具是 `setuptools` 程式庫的一部分，用於讓內建第三方程式庫存取隨附的程式庫。更新 `appengine_config.py`，使用 `pkg_resources` 將其指向 `lib` 中的隨附程式庫。完成這項變更後，整個檔案應如下所示：

```
import pkg_resources
from google.appengine.ext import vendor

# Set PATH to your libraries folder.
PATH = 'lib'
# Add libraries installed in the PATH folder.
vendor.add(PATH)
# Add libraries to pkg_resources working set to find the distribution.
pkg_resources.working_set.add_entry(PATH)
```

5. 更新應用程式檔案

完成設定檔的相關手續後，您現在可以從 `ndb` 遷移至 Cloud NDB。如要完成遷移，請更新匯入的程式庫，並在 `main.py` 中新增背景資訊管理功能。

1. 匯入

在 `main.py` 中進行下列匯入項目交換：

- 更新前

```
from google.appengine.ext import ndb
```

- AFTER：

```
from google.cloud import ndb
```

有時從 App Engine 程式庫改用 Google Cloud 程式庫，就像這個例子一樣，只會出現細微的差異。如果內建服務已成為完整的 Google Cloud 產品，您將從 `google.cloud` 匯入屬性，而非 `google.appengine`。

2. Datastore 存取權

如要使用 Cloud NDB 程式庫，應用程式必須使用 [Python 內容管理員](#)。其目的是「控管」資源存取權，使用者必須先取得資源才能使用。情境管理員是以電腦科學控制技術為基礎，也就是[資源配置即初始化 \(RAII\)](#)。內容管理員會與 Python 檔案 (必須先開啟才能存取) 和並行作業搭配使用，且必須先取得「[自旋鎖](#)」，才能執行「[重要區段](#)」中的程式碼。

同樣地，Cloud NDB 也要求您先取得用戶端的環境，才能與 Datastore 通訊，之後才能執行任何 Datastore 指令。首先，請在 `main.py` 中緊接在 [Flask 初始化](#) 之後新增 `ds_client = ndb.Client()`，建立用戶端 (`ndb.Client()`)：

```
app = Flask(__name__)
ds_client = ndb.Client()
```

Python `with` 指令僅用於取得物件的內容。使用 `with` 陳述式包裝存取 Datastore 的任何程式碼區塊。

以下是第 1 堂課程中的相同函式，用於將新實體寫入 Datastore，以及讀取並顯示最近新增的實體：

- 更新前：

以下是沒有內容管理功能的原始程式碼：

```
def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    Visit(visitor='{:}: {}'.format(remote_addr, user_agent)).put()

def fetch_visits(limit):
    'get most recent visits'
    return (v.to_dict() for v in Visit.query().order(
        -Visit.timestamp).fetch(limit))
```

- AFTER：

現在新增 `with ds_client.context():`，並將 Datastore 存取程式碼移至 `with` 區塊：

```
def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    with ds_client.context():
        Visit(visitor='{ }: {}'.format(remote_addr, user_agent)).put()

def fetch_visits(limit):
    'get most recent visits'
    with ds_client.context():
        return (v.to_dict() for v in Visit.query().order(
            -Visit.timestamp).fetch(limit))
```

警告：與 `ndb` 不同，使用交易和 Cloud NDB 時，工作佇列會獨立運作。舉例來說，在交易程式碼中，如果交易失敗，工作佇列呼叫不會回溯，只有 Datastore 作業會回溯。雖然我們沒有這項限制的相關文件，但 Cloud Tasks 頁面的「不適用」部分[列出了這項限制](#)。

主要驅動程式應用程式與單元 1 中的內容相同，因為這裡沒有 `ndb` (也沒有 Cloud NDB) 程式碼：

```
@app.route('/')
def root():
    'main application (GET) handler'
    store_visit(request.remote_addr, request.user_agent)
    visits = fetch_visits(10)
    return render_template('index.html', visits=visits)
```

最佳做法是確保應用程式程式碼和資料存取權之間有明確區別。這樣一來，當基礎資料儲存機制變更時 (如本次遷移作業)，您的主要應用程式程式碼就不會受到影響。

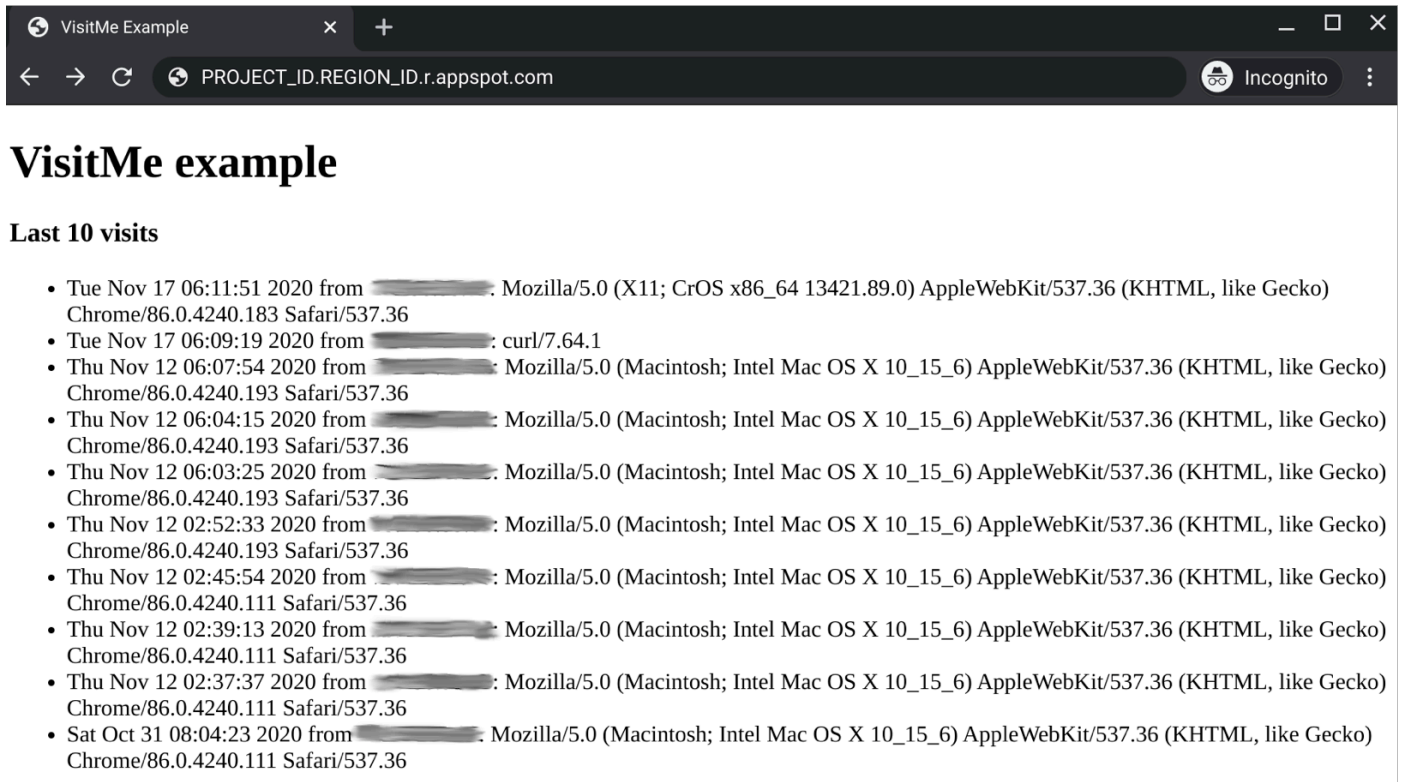
步驟 6 · 約 2 分鐘

6. 摘要/清除

部署應用程式

使用 `gcloud app deploy` 重新部署應用程式，並確認應用程式是否正常運作。您的程式碼現在應與第 2 堂課的存放區內容相符。

如果您直接跳到本系列課程，而未完成任何先前的程式碼研究室，應用程式本身不會變更，但會記錄所有對主要網頁 (/) 的造訪，且在您造訪網站足夠次數後，會顯示如下：



VisitMe example

Last 10 visits

- Tue Nov 17 06:11:51 2020 from [redacted]: Mozilla/5.0 (X11; CrOS x86_64 13421.89.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.183 Safari/537.36
- Tue Nov 17 06:09:19 2020 from [redacted]: curl/7.64.1
- Thu Nov 12 06:07:54 2020 from [redacted]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.193 Safari/537.36
- Thu Nov 12 06:04:15 2020 from [redacted]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.193 Safari/537.36
- Thu Nov 12 06:03:25 2020 from [redacted]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.193 Safari/537.36
- Thu Nov 12 02:52:33 2020 from [redacted]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.193 Safari/537.36
- Thu Nov 12 02:45:54 2020 from [redacted]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.111 Safari/537.36
- Thu Nov 12 02:39:13 2020 from [redacted]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.111 Safari/537.36
- Thu Nov 12 02:37:37 2020 from [redacted]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.111 Safari/537.36
- Sat Oct 31 08:04:23 2020 from [redacted]: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/86.0.4240.111 Safari/537.36

恭喜您完成本第 2 模組程式碼研究室。恭喜您完成最後一項強烈建議的 Datastore 遷移作業，

選用：清除

在準備進行下一個遷移程式碼研究室之前，請先清除資源，以免產生帳單費用。身為現有開發人員，您可能已熟悉 [App Engine 的價格資訊](#)。

重要事項：原始 App Engine 執行階段不需要有效的帳單帳戶，但[現在已不再是如此](#) (2019 年 11 月開始，2020 年 4 月開始強制執行)。不過請放心，App Engine 仍提供[免費方案](#)，只要不超出限制，就不會產生任何費用。

選用：停用應用程式

如果還沒準備好進行下一個教學課程，請[停用應用程式](#)，以免產生費用。準備好進行下一個程式碼研究室時，可以重新啟用這項功能。應用程式停用後，不會產生任何流量，因此不會產生費用。不過，如果[資料儲存空間用量](#)超出[免費配額](#)，您仍須支付相關費用，因此請刪除足夠的資料，確保用量低於上限。

另一方面，如果您不打算繼續遷移，並想完全刪除所有內容，可以[關閉專案](#)。

後續步驟

接下來的行動可自由選擇。選擇下列任一選項：

- **單元 2 額外內容：**繼續閱讀本教學課程的額外部分，瞭解如何移植到 Python 3，以及下一代 App Engine 執行階段。
 - **單元 7：**App Engine 發送工作佇列 (如果您使用 [發送] 工作佇列，則為必填)
 - 將 App Engine `taskqueue` 推送工作新增至模組 1 應用程式
 - 為第 8 單元中遷移至 [Cloud Tasks](#) 的作業做好準備
 - **單元 4：**使用 Docker 遷移至 Cloud Run
 - 使用 Docker 將應用程式容器化，以便在 Cloud Run 上執行
 - 可讓您繼續使用 Python 2
 - **單元 5：**使用 Cloud Buildpacks 遷移至 Cloud Run
 - 使用 Cloud Buildpacks 將應用程式容器化，以便在 Cloud Run 上執行
 - 您不需要瞭解 Docker、容器或 `Dockerfile`s
 - 您必須先將應用程式遷移至 Python 3
 - **單元 3：**
 - 將 Cloud NDB 的 Datastore 存取權現代化為 Cloud Datastore
 - 這是用於 Python 3 App Engine 應用程式和非 App Engine 應用程式的程式庫
-

7. 加分題：遷移至 Python 3

如要使用最新的 App Engine 執行階段和功能，建議您遷移至 Python 3。在我們的範例應用程式中，Datastore 是我們使用的唯一內建服務，而且由於我們已從 `ndb` 遷移至 Cloud NDB，因此現在可以移植到 App Engine 的 Python 3 環境。

總覽

雖然遷移至 Python 3 並非 Google Cloud 教學課程的範圍，但本程式碼研究室的這部分可讓開發人員瞭解 Python 3 App Engine 執行階段的差異。新一代執行階段的一項出色功能是簡化第三方套件的存取權，您不需要在 `app.yaml` 中指定內建套件，也不需要複製或上傳「非」內建程式庫，只要列在 `requirements.txt` 中，系統就會隱含安裝。

由於我們的範例非常基本，且 Cloud NDB 與 Python 2-3 相容，因此不必將任何應用程式碼明確移植到 3.x；應用程式可在 2.x 和 3.x 上執行，且無需修改，也就是說，在這種情況下，唯一需要變更的是設定：

1. 簡化 `app.yaml`，以參照 Python 3 並移除第三方程式庫。
2. 刪除 `appengine_config.py` 和 `lib` 資料夾，因為不再需要這些項目。

除了 `main.py` 之外，`requirements.txt` 和 `templates/index.html` 檔案維持不變。

簡化 `app.yaml`

更新前：

這個範例應用程式的唯一實際變更，就是大幅縮短 `app.yaml`。提醒您，以下是 [app.yaml 第 2 單元](#)結束時的內容：

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app

libraries:
- name: grpcio
  version: 1.0.0
- name: setuptools
  version: 36.6.0
```

AFTER：

在 Python 3 中，`threadsafe`、`api_version` 和 `libraries` 指令已全數淘汰；所有應用程式都會假設為執行緒安全，且 Python 3 不會使用 `api_version`。App Engine 服務不再預先安裝內建的第三方套件，因此 `libraries` 也已淘汰。如要進一步瞭解這些異動，請參閱 [app.yaml 變更說明文件](#)。因此，您應從 `app.yaml` 刪除這三個檔案，並更新至支援的 Python 3 版本 (請參閱下文)。

選用：使用 `handlers` 指令

此外，引導 App Engine 應用程式流量的 `handlers` 指令也已淘汰。由於新一代執行階段會要求網頁架構管理應用程式路徑，因此所有「處理常式指令碼」都必須變更為「`auto`」。結合上述變更後，您會得到以下 `app.yaml`：

```
runtime: python38

handlers:
- url: /*
  script: auto
```

如要進一步瞭解 `script: auto`，請參閱[說明文件頁面](#)。

正在移除 `handlers` 指令

由於 `handlers` 已淘汰，您也可以移除整個區段，只留下單行 `app.yaml`：

```
runtime: python38
```

根據預設，這會啟動 [Gunicorn WSGI 網頁伺服器](#)，適用於所有應用程式。如果您熟悉 `gunicorn`，這是以基本 `app.yaml` 預設啟動時執行的指令：

```
gunicorn main:app --workers 2 -c /config/gunicorn.py
```

選用：使用 `entrypoint` 指令

不過，如果應用程式需要特定啟動指令，可以使用 `entrypoint` 指令指定，其中 `app.yaml` 會如下所示：

```
runtime: python38
entrypoint: python main.py
```

這個範例特別要求使用 Flask 開發伺服器，而非 `gunicorn`。您也必須在應用程式中加入啟動開發伺服器的程式碼，才能透過通訊埠 8080 在 `0.0.0.0` 介面上啟動，方法是在 `main.py` 底部加入這個小節：

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

如要進一步瞭解 `entrypoint`，請參閱[說明文件頁面](#)。如需更多範例和最佳做法，請參閱 [App Engine 標準環境啟動說明文件](#)和 [App Engine 彈性環境啟動說明文件](#)。

刪除 `appengine_config.py` 和 `lib`

刪除 `appengine_config.py` 檔案和 `lib` 資料夾。遷移至 Python 3 時，App Engine 會取得並安裝 `requirements.txt` 中列出的套件。

`appengine_config.py` 設定檔可用於辨識第三方程式庫/套件，無論您是自行複製，還是使用 App Engine 伺服器上已有的程式庫/套件 (內建)，都適用。改用 Python 3 時，主要變更摘要如下：

1. 不會將複製的第三方程式庫 (列於 `requirements.txt`) 組合在一起
2. 沒有 `pip install` 進入 `lib` 資料夾，也就是沒有 `lib` 資料夾週期
3. `app.yaml` 中未列出內建第三方程式庫
4. 不必參照應用程式至第三方程式庫，因此沒有 `appengine_config.py` 檔案

您只需要在 `requirements.txt` 中列出所有必要的第三方程式庫。

部署應用程式

重新部署應用程式，確保應用程式正常運作。您也可以確認解決方案與[單元 2 的 Python 3 程式碼範例](#)有多接近。如要以視覺化方式呈現 Python 2 的差異，請比較程式碼與其 [Python 2 版本](#)。

恭喜你完成第 2 單元的加分步驟！請參閱[這份文件](#)，瞭解如何準備 Python 3 執行階段的設定檔。最後，請查看 (先前的)「摘要/清除」頁面，瞭解後續步驟和清除作業。

準備您的應用程式

當您要遷移應用程式時，必須將 `main.py` 和其他應用程式檔案移植到 3.x，因此最佳做法是盡量讓 2.x 應用程式「向前相容」。

網路上有許多資源可協助您達成目標，但以下是一些重要提示：

1. 確認所有應用程式依附元件完全相容於 3.x 版
2. 確認應用程式至少執行 2.6 版 (最好是 2.7 版)
3. 確保應用程式通過整個測試套件 (且涵蓋率至少達 80%)
4. 使用相容性程式庫，例如 `six`、Future 和/或 Modernize
5. 瞭解 2.x 與 3.x 的主要差異 (不相容於舊版)
6. 任何 I/O 都可能導致 Unicode 與位元組字串不相容

設計範例應用程式時，我們已將上述事項納入考量，因此應用程式可直接在 2.x 和 3.x 上執行，方便我們專注於向您說明使用新一代平台時需要變更的內容。

8. 其他資源

App Engine 遷移模組程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

下表提供模組 1 (START) 和模組 2 (FINISH) 的存放區資料夾連結。您也可以從[所有 App Engine Codelab 遷移作業的存放區](#)存取這些範例，並複製或下載 ZIP 檔案。

Codelab	Python 2	Python 3
Module 1	code	(不適用)
Module 2	code	code

App Engine 資源

以下是與這項特定遷移作業相關的其他資源：

- Python NDB 參考資料
 - [App Engine](#) `ndb` 文件
 - [App Engine](#) `ndb` repo
 - [Google Cloud NDB 說明文件](#)
 - [Google Cloud NDB 存放區](#)
- (舊版) 從 Python 2.5 和 `webapp` 遷移至 2.7 和 `webapp2`
 - [Python 2.5 已於 2013 年淘汰，並於 2017 年停止支援，改用 2.7 版](#)
 - [Python 2.5 至 2.7 文件](#)
 - [Python 2.5](#) `db` 程式庫
 - [從 Python 2.5](#) `db` 遷移至 2.7 `ndb`
- 遷移至 Python 3 和 GAE 次世代執行階段
 - [從 GAE](#) `ndb` 遷移至 Cloud NDB
 - [GAE](#) `ndb` 遷移至 Cloud NDB 說明文件範例存放區
 - [指令碼處理常式為選用項目](#)
 - [entrypoint](#) 參考資料
 - [app.yaml](#) 範例和最佳做法
 - [更多範例和最佳做法](#)
 - [遷移 App Engine 設定檔](#)

- [遷移作業首頁](#)
 - 一般
 - [在 Google Cloud Platform 上執行 Python](#)
 - [Google Cloud Python 用戶端程式庫](#)
-